

Project: Cryptography - Chef's Corner

A cybersteps project

Date: 5. December 2025

Participants: Gregor

1. Introduction

This project's objective was to analyze a provided website `chefscorner.cybersteps.de` and identify vulnerabilities that could allow an attacker to extract sensitive information, specifically the *secret recipe*.

The assessment involved examining the application's behavior, reviewing publicly available source code, crafting forged authentication tokens, and ultimately decrypting the secret recipe.

2. Initial Reconnaissance

2.1 Website Inspection

The first step was to manually inspect the website's structure and visible components. During this process, I discovered:

- An **admin login section** with a note about *default credentials*.
- A **GitHub repository link** in the page footer, referencing the full source code of the Flask application.

Both findings were promising attack vectors.

2.2 GitHub Source Code Review

The publicly accessible GitHub repository contained all logic for login, JWT token creation, API authorization, encryption routines, and more. This allowed a detailed analysis of how authentication and access control mechanisms were intended to work.

3. Exploiting the Admin API

3.1 Understanding JWT Authentication

The repository contained a private/public RSA key pair, crucial for generating valid admin JWT tokens. The login route showed how these tokens were created:

```
# Create JWT token
payload = {
    'username': username,
    'role': 'admin',
    'exp': 9999999999 # Far future expiration - TODO: Set
proper expiration
}
token = jwt.encode(payload, PRIVATE_KEY, algorithm='RS256')
```

Because the **private key** was committed to **GitHub**, I could generate my own valid admin token.

3.2 Accessing the /api/config Endpoint

The endpoint /api/config checks for admin-level JWT tokens.

After generating a forged token, I intercepted a request and added the header:

```
Authorization: Bearer <generated_token>
```

Sending this request successfully returned the server's internal configuration, including the critical **server seed**, which later proved essential.

Request	Response
<pre>1 GET /api/config HTTP/2 2 Host: chefscorner.cybersteps.de 3 Cookie: _gcl_au=1.1.1559412968.1764156840; _cfuvid= gegKGLM4AZkH.777n5Fe7W04Sa_Xxs1EpuH.9uGPRly-1764927304615-0.0.1.1-604800000; __cf_bm= eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWwluIiwicm9sZSI6ImFkbWwluIiwiaXNwIj oxMDI0fQ.eyJ1c2VybmFtZSI6ImFkbWwluIiwicm9sZSI6ImFkbWwluIiwiaXNwIj JrkbUBS3DMt0Wnu1Z7eKry1uZfJcWpVx60RBQYKwD0my-lqeh4r2oHzsaICzYpFmwYd3aQhrrieLS-E_hMhmQmHyeb wchPdMzA-BXz-2TrIs9uVWMAzhwy__Z2GFMM_mEazqPLfRbgXRE0xT9HFFH5zILiV4smys7Wxsm509kylB6wyVfueh NvhXEcKb52efsJhC8YI-s3rtBIwMCAW0sfulsSF13G0zhQ2mGxQhCKx1QtD5xeWXDhdFEr0VhXMdw 4 Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWwluIiwicm9sZSI6ImFkbWwluIiwiaXNwIj o5OTk5OTk5OTk5fQ.on7qQatN6hPt0s6uR1149AkYs0Wxs2Lc6j2w5PcxVCR2N_CcoHdPWuRuCi9ky1KpYVt42CP0 RzjM101pmeJl_BT0j_9q9l3XrFE18C1e6KuxEz3g5j23bb0wbx09v10ou46MEFQ1uRpEsZa_F1UgkMoqJy2Mht2eR dFbsuOKP2E1tVvOpDmMhZ2eTqC18555x9qilL6BQfBnykfxoG1P9EB01lx0E4Cn5ZBCxtpt416w6o7shdGoIHBxPO 16GdL1ZISUbdJxVYPZgZ7o2e3eQwUtIj8CwDZvZhbZU18nG1G5qRPLzg34I-QLKcYs0yBrhZk5xjYzDf7CPsYA 5 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:145.0) Gecko/20100101 Firefox/145.0 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 7 Accept-Language: de,en-US;q=0.7,en;q=0.3 8 Accept-Encoding: gzip, deflate, br 9 Upgrade-Insecure-Requests: 1 10 Sec-Fetch-Dest: document 11 Sec-Fetch-Mode: navigate 12 Sec-Fetch-Site: none 13 Sec-Fetch-User: ?1 14 Priority: u=0, i 15 Te: trailers 16 17</pre>	<pre>1 HTTP/2 200 OK 2 Server: Vercel 3 X-Vercel-Id: pPvGx5x5W8TvPdyPiwbCD23LKK5BmgUq 4 Cache-Control: public, max-age=0, must-revalidate 5 Content-Type: application/json 6 Date: Fri, 05 Dec 2025 09:59:47 GMT 7 Server: Vercel 8 Strict-Transport-Security: max-age=63072000 9 X-Vercel-Cache: BYPASS 10 X-Vercel-Id: fra1::lad1::rm6df-1764928787594-24b2d24b1c12 11 Content-Length: 92 12 13 { 14 "environment": "production", 15 "server_seed": "super-secret-server-seed-456", 16 "version": "1.0.0" 17 }</pre>

4. Extracting the Secret Recipe

4.1 Reverse-Engineering the Recipe Lookup

Recipes could be retrieved through:

```
/api/recipe/<recipe_id>
```

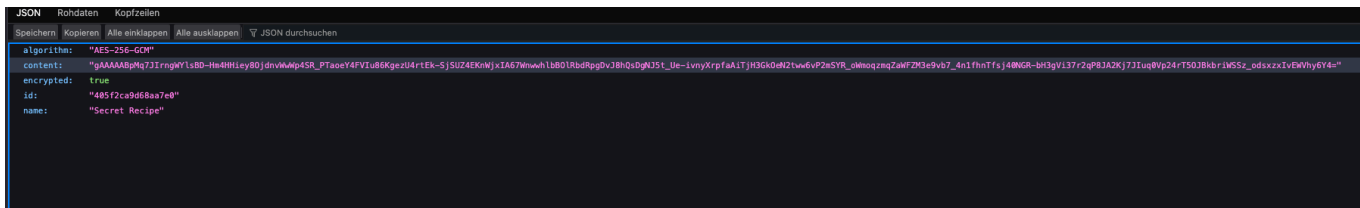
The source code showed that the recipe ID was derived from:

```
combined = f"{SERVER_SEED}:{recipe_name}"

print(hashlib.sha256(combined.encode()).hexdigest()[:16])
```

The recipe name for the secret sauce was visible on the website: **SECRETSAUCE**.

Using the leaked server seed and recipe name produced the recipe ID (405f2ca9d68aa7e0) and allowed retrieving the *encrypted* secret recipe.



5. Decrypting the Secret Recipe

5.1 Analyzing the Encryption

The application used Fernet (AES-based) encryption with the following routine:

```
ENCRIPTION_KEY = os.environ.get('ENCRIPTION_KEY', 'default-encryption-key')

def encrypt_with_key(data, key):
    """AES encryption using the provided key"""
    from cryptography.fernet import Fernet
    from cryptography.hazmat.primitives import hashes
    from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC

    # Derive key from password using PBKDF2
    key_bytes = key.encode() if isinstance(key, str) else key
    salt = b'chefs_corner_salt' # Using fixed salt for consistency - should
    be fine

    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        iterations=100000,
    )
    derived_key = base64.urlsafe_b64encode(kdf.derive(key_bytes))
```

```
fernet = Fernet(derived_key)

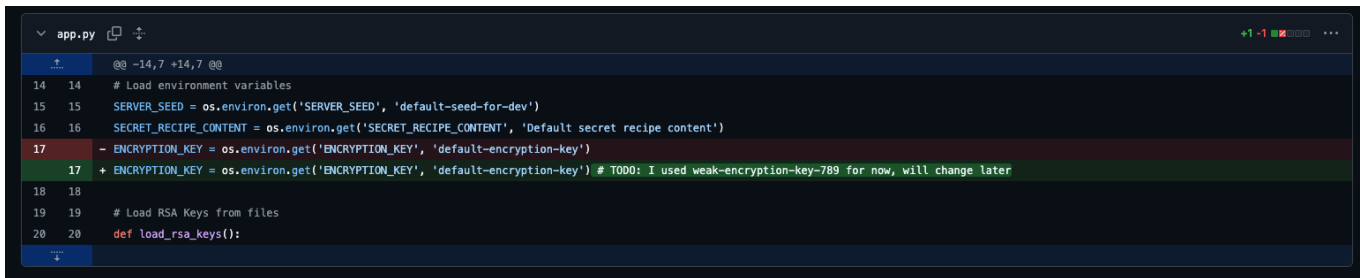
data_bytes = data.encode() if isinstance(data, str) else data
return fernet.encrypt(data_bytes).decode()
```

5.2 Key Discovery

I tried using the fallback key or the server seed. Both failed.

Further inspection of the GitHub commit history revealed a developer comment indicating a temporary key:

"weak-encryption-key-789"

A screenshot of a code editor window titled 'app.py'. The code is in Python and shows the loading of environment variables and the handling of an encryption key. Line 17 is highlighted in green, showing a comment about using a weak encryption key. The code includes comments in Chinese and English.

```
14 14 # Load environment variables
15 15 SERVER_SEED = os.environ.get('SERVER_SEED', 'default-seed-for-dev')
16 16 SECRET_RECIPE_CONTENT = os.environ.get('SECRET_RECIPE_CONTENT', 'Default secret recipe content')
17 17 - ENCRYPTION_KEY = os.environ.get('ENCRYPTION_KEY', 'default-encryption-key')
17 17 + ENCRYPTION_KEY = os.environ.get('ENCRYPTION_KEY', 'default-encryption-key') # TODO: I used weak-encryption-key-789 for now, will change later
18 18
19 19 # Load RSA Keys from files
20 20 def load_rsa_keys():
```

Using this key with the decryption function successfully produced the plaintext secret recipe:

“The secret sauce recipe: 1 kilo practice, 1/2 kilo fun, a pinch of curiosity, a handful of mistakes, stirred with persistence.”

6. Additional Vulnerability: Broken Admin Login

Despite the hardcoded admin credentials (admin / admin123), the login form did not work.

Analysis showed:

- The backend expected application/x-www-form-urlencoded
- The frontend submitted application/json

By manually sending a request with the correct content type, authentication worked as intended, confirming a broken login implementation.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST /login HTTP/2				1 HTTP/2 200 OK			
2 Host: cheffscorner.cybersteps.de				2 Cache-Control: public, max-age=0, must-revalidate			
3 Cookie: _gcl_au=1.1.1559412968.1764156840; _cfuvid=.2ubDaRseYE8Ma1JWtk4IYwKyeXcwZmR15dN78tw8-1764921642770-0.0.1.1-604800000				3 Content-Type: application/json			
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:145.0) Gecko/20100101 Firefox/145.0				4 Date: Fri, 05 Dec 2025 13:50:41 GMT			
5 Accept: */*				5 Server: Vercel			
6 Accept-Language: de,en-US;q=0.7,en;q=0.3				6 Set-Cookie: session=.eJxdisuWaaAAP8l-5mjBRmzG-2RFuIRhN7KDJFoRJmMmjn33---s7uIuaINVP5uRnmW3-dyU113lVke8Yp_1FYEJQQX1oVxApKcmTNoq---_EYjEZMkf5ohPv-FhNm5nRCZSffQ2f80p9WMap7Uf0PW_B0u900fEedUvniYsoUAiCpX02aMehMmegUSi30zIc8gaF4vMCC7q0rfw4nCq1WdpDnyWjJELfC9fatQXSP4w6vuSRUF_UQXW26dM0nA8ISktJp6exbu9yJbd5Hsw5f9RXQr1JULxUmKE7hb3BML2YJ84aLw52trruKJbh1cz5B1s3yBj8tJ7_dutsFhgAyQfs1xz5B0q21BqVXLkd84tR0xvUMTwsIFQ4KdJDW48rGXdIrdMeqq0XRXu0-B6y73mJwinPd5kmm4zveUbhcl5Ecj6jW4KzjF97n0AZaZ7WMPiR-u963EnoLnEORZcxbtGe1NjKvs1W_7aB_yr42f_4CR9eb7g.aTLJMQ.B97mnvS1IRjzCjy2r3d5zJQbLg;			
7 Accept-Encoding: gzip, deflate, br				7 Strict-Transport-Security: max-age=63072000			
8 Referer: https://cheffscorner.cybersteps.de/login				8 Vary: Cookie			
9 Content-Type: application/x-www-form-urlencoded				9 X-Vercel-Cache: MISS			
10 Content-Length: 32				10 X-Vercel-Id: fra1:iad1::85b45-1764942641030-f49d6c52e689			
11 Origin: https://cheffscorner.cybersteps.de				11 Content-Length: 478			
12 Sec-Fetch-Dest: empty							
13 Sec-Fetch-Mode: cors							
14 Sec-Fetch-Site: same-origin							
15 Priority: u=0							
16 Te: trailers							
17							
18 username=admin&password=admin123				13 {			
				"success":true,			
				"token":			
				"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWw1Iiwicm9sIjoiZmFkbWw1IiwiaWF0Ij01OTk5OTk5OTk5fQ.on7QaTn6hPIG6uR1149AKYs0Wxs2Lc6j2wsPCxVCR2N_CcoHTdPWRuCi9hky1KpYVt42CPORzjMJ0ipmEjU_bT0j_9q9L3XrFEL8C1eG6XurEz3g5jZ3bbQwbx09vVo0u46MIefQLuRpEsZo_FiUqMoaQJYzMMtZeRdFbsuOKJPEitVv0pDaMhZ2eTqC18555x9qiL16BQFBhyNxfx0GI1P9EB01lx0E4Cn5ZBCzxtP416w6o7shdGoIHBxP01GgDLZ1SubbJxVYPZg7z02e3eQwUt1jBCw0Zv2HbZU1BnGLG5qRP1zg341-QLKCYyS0yBrhZk5xjYz0f7CPSYA"			
				14 }			

7. Summary of Identified Vulnerabilities

7.1 High-Risk Issues

- **Private RSA keys committed to GitHub**, enabling arbitrary JWT forgery.
- **Weak and hardcoded encryption key**, exposed in repository history.
- **Hardcoded salt** used in PBKDF2.
- **Server seed exposure** via misconfigured admin API.
- **Encryption and signing keys not separated**, violating cryptographic best practices.

7.2 Medium & Low-Risk Issues

- **Broken login implementation** due to mismatched content types.
- **Environment variable fallbacks to insecure defaults** (JWT keys, encryption keys).
- **Sensitive keys persisted in Git history**, even after changes.
- **Same key and method used for multiple cryptographic purposes** (signing + encryption).

8. Recommendations

Authentication & Authorization

- Remove all RSA key pairs from the repository.
- Regenerate new private/public keys securely.
- Enforce strict validation of JWT signatures.
- Implement role-based access control with proper key isolation.

Cryptography

- Never use hardcoded salts; load them from environment variables.
- Use distinct keys for signing and encryption.
- Remove fallback cryptographic defaults.
- Rotate all keys exposed in Git history.

Infrastructure & Configuration

- Disable access to sensitive configuration endpoints.
- Ensure environment variables are properly loaded at runtime.

Code Quality & Security Hygiene

- Fix the frontend-backend mismatch for the admin login form.
- Audit commit history and purge sensitive information.
- Add automated secret scanning to the CI/CD pipeline.
- Avoid storing any sensitive values inside the public repo.

9. Conclusion

Through a combination of source code analysis, misconfiguration exploitation, JWT forgery, and cryptographic key misuse, it was possible to gain full administrative access, derive the recipe ID, retrieve the encrypted secret recipe, and ultimately decrypt it.

The project highlights the importance of:

- avoiding hardcoded secrets
- securing cryptographic assets
- validating authentication flows
- removing sensitive information from version control

This exercise demonstrates how small oversights — such as a leaked key or insecure fallback — can chain together into a full compromise of an application.